

Cuckoo Hashing with Bounded Displacement for Exact-Match Flow Tables

Soumyajit Kolay
School of Computer Engineering
KIIT Deemed University
Bhubaneswar, India
2650026@kiit.ac.in

Abstract—This paper deals in discussing a variant of d -ary Cuckoo Hashing Tables that considers bounded displacement to ensure a higher value for recommended load, on which it does with lesser number of full rehashes as compared to previous algorithms. This paper is still in progress and serves as a $\text{\LaTeX}$ learning playground for the author.

Index Terms—Software-Defined Networking, Flow Tables, Cuckoo Hashing, Bounded Displacement, Stash Mechanisms, Deterministic Latency, Line-Rate Packet Processing

I. INTRODUCTION

Para 1 — “Exact-Match Flow Tables.” line-rate dictionary requirement; fixed per-packet time budget

A. Literature Review

Para 2 — “Classical Cuckoo Hashing.” Pagh-Rodler; 2-way displacement; $O(1)$ worst-case lookup

Para 3 — “The Full-Rehash Catastrophe.” $\text{MaxLoop} = \Theta(\log n)$; $O(n)$ full rehash; why that’s unacceptable at line rate

Para 4 — “Stash-Based Mitigation.” (chronological) Kirsch/Kutzelnigg/Aumüller/Fountoulakis-Panagiotou — reduces rehash odds, but all still assume unbounded search before stash

Para 5 — “Networking-Specific Variants.” (chronological) Pontarelli x2/Le Scouarnec/Kuszmaul/Noble/Minaud-Papamanthou/TECache — throughput/energy-focused, no worst-case insertion bound

[one-line forward pointer: Sec. II quantifies baseline rehash frequency first]

B. Implementation-Driven Corrections

Building a reference implementation surfaced one correctness issue not apparent from the pseudocode alone. The initial REHASHSTASH design redrew a fresh global hash-function tuple on every stash overflow and reinserted only the stashed key(s). This silently orphaned every key already resident in the main tables: their physical slot had been computed under the *old* hash functions, so a lookup issued under the new ones would almost never find them. The anomaly first appeared as an implausible positive-lookup probe-count distribution rather than as an outright failure, and was only isolated with a standalone occupancy diagnostic that showed the physical key distribution contradicted the reported lookup behavior. The

corrected design keeps the global hash functions fixed for the table’s lifetime and instead retries the stashed key’s bounded-kick placement with fresh *randomized eviction choices* only – simpler than the original design, and it removes the hazard entirely while preserving the $O(s)$ incremental-rehash cost. A small recursion-depth guard was added defensively against pathological repeated-retry cases, though it was never observed to trigger.

A second, lower-level lesson concerned evaluation honesty rather than correctness: a single benchmark point near Pagh-Rodler’s own conservative recommended load ($\alpha \approx 1/3$) shows essentially zero full rehashes for the classical baseline, which would understate the exact failure mode this paper is motivated by. Only sweeping load factor toward the $d=2$ structural ceiling ($\alpha \approx 0.495$) surfaces the real tail blow-up, indicating that near-ceiling operating points must be tested explicitly rather than only “typical” ones.

C. Contributions

Para 7 — “Contributions.” bulleted, each tagged to its section.

II. MODELING REHASH FREQUENCY IN CLASSICAL CUCKOO HASHING

Before presenting the proposed design, this section quantifies the failure mode it targets. In classical 2-way cuckoo hashing, an insertion’s eviction loop is bounded by $\text{MaxLoop} = \lceil 3 \log_{1+\varepsilon} r \rceil$, where $r = (1 + \varepsilon)n$ is the table capacity [1]. Pagh and Rodler show the probability that a single insertion exhausts MaxLoop – and therefore triggers a full-table rehash – is bounded by

$$\Pr[\text{rehash triggered}] \leq 2(1 + \varepsilon)^{-\frac{2 \cdot \text{MaxLoop} - 1}{3} + 1} + O(n^{-2}), \quad (1)$$

the same path-counting bound used later in Section IVB, evaluated here at $t = \text{MaxLoop}$ instead of a fixed constant. Because MaxLoop grows with n , this probability is small *in the asymptotic limit* – but ε is fixed by the table’s slack at construction time, and for a given finite n the realized rehash frequency depends heavily on how close the load factor $\alpha = n/r$ sits to the d -ary structural ceiling (e.g. $\approx 50\%$ for $d = 2$ [1]). Eq. 1 predicts that rehash frequency should rise sharply as α approaches that ceiling, independent of n ’s asymptotic comfort.

This prediction is checked directly against a reference implementation in Section VI: TBD (*technique*: measured directly from the classical-cuckoo baseline in `bench.cpp` at two operating points – Pagh and Rodler’s own recommended $\alpha \approx 1/3$, and a point near the $d=2$ ceiling, $\alpha \approx 0.49$ – 0.495 ; see Section VIB). As with the modeling-before-design pattern this section follows, Eq. 1 is not claimed to predict exact rehash counts under correlated or adversarial insertion orderings – it motivates *why* full-table rehashing is the right failure mode to target, and gives a closed-form baseline the proposed scheme’s own bounds (Section IV) are compared against. The remainder of the paper addresses this failure mode structurally rather than by tuning ε or MaxLoop alone (see the Discussion, Section VII, for why tuning MaxLoop down is not by itself a solution).

III. SYSTEM ARCHITECTURE & PROPOSED ALGORITHM

A. Design Goals

Bounding the eviction chain and introducing a stash is a structural departure from classical cuckoo hashing, and that departure introduces four concerns that did not exist in the unbounded scheme. The remainder of this paper is organized around resolving each:

- G1 Bounded worst-case latency.** Every insertion must complete in time independent of n (Section IVA).
- G2 Rehash correctness.** Giving a stashed key a fresh placement attempt must never silently break lookups for keys already resident in the main tables (Section IIIC, Section VII).
- G3 Bounded stash pressure.** MaxKicks and stash capacity s must be sized jointly against a quantified stash-routing probability, not chosen arbitrarily (Section IVB–C, Section VID).
- G4 Load-factor coupling.** MaxKicks, s , and the d -ary load threshold are not independent knobs; operating near the hypergraph’s structural ceiling changes the whole routing-probability profile (Section IVD).

B. Formal Mathematical Definitions & System Model

Let $U = \{0, 1\}^w$ be the universe of w -bit flow identifiers (e.g., standard IPv4 5-tuples) [1]. An exact-match flow table consists of $d \geq 2$ independent sub-tables $T_1, \dots, T_d$, each containing m slots [1]. The total capacity of the main table is $r = dm$ slots. The table maintains $n = |S|$ active microflows from a set $S \subset U$. The overall load factor is

$$\alpha = \frac{n}{r} = \frac{n}{dm}. \quad (2)$$

The dictionary uses d uniform, independent hash functions drawn from a universal hash family $\mathcal{H}$:

$$h_i : U \rightarrow \{0, 1, \dots, m-1\}, \quad \forall i \in \{1, \dots, d\}. \quad (3)$$

Every stored flow key $x \in S$ resides either in one of its d candidate sub-table locations $T_i[h_i(x)]$ or in an auxiliary stash Stash of capacity $s \ll n$ [2]. Empty table slots are denoted $\perp$. Hash functions $h_1, \dots, h_d$ are fixed for the table’s lifetime once any key has been placed under them (see Algorithm 3 and G2 above).

Algorithm 1 LOOKUP(x)

```

1: for  $i \leftarrow 1$  to  $d$  do
2:   if  $T_i[h_i(x)] = x$  then
3:     return FOUND
4:   end if
5: end for
6: for each entry  $e$  in STASH do
7:   if  $e = x$  then
8:     return FOUND
9:   end if
10: end for
11: return NOTFOUND

```

Algorithm 2 INSERT(x)

```

1: if LOOKUP( $x$ ) = FOUND then
2:   return ALREADYPRESENT
3: end if
4: if  $(n+1)/(dm) > \text{LOADTHRESHOLD}(d)$  then  $\triangleright$  [3]
5:   RESIZE()
6: end if
7:  $cur \leftarrow x$ 
8: for  $round \leftarrow 1$  to MaxKicks do  $\triangleright$  constant, independent of  $n$ 
9:   for  $i \leftarrow 1$  to  $d$  do
10:    if  $T_i[h_i(cur)] = \perp$  then
11:       $T_i[h_i(cur)] \leftarrow cur$ ;  $n \leftarrow n+1$ 
12:      return SUCCESS
13:    end if
14:   end for
15:    $i \leftarrow \text{CHOOSECANDIDATEINDEX}(cur)$   $\triangleright$  uniform random
16:    $evicted \leftarrow T_i[h_i(cur)]$ ;  $T_i[h_i(cur)] \leftarrow cur$ 
17:    $cur \leftarrow evicted$ 
18: end for
19: if STASH has a free slot then
20:   place  $cur$  in STASH;  $n \leftarrow n+1$ 
21:   return SUCCESS  $\triangleright$  via stash
22: end if
23: REHASHSTASH()  $\triangleright$  incremental — Alg. 3
24: return INSERT( $x$ )  $\triangleright$  retry; depth-guarded, see Alg. 3

```

C. Algorithmic Specifications

Two design points distinguish Algorithm 2 from the classical procedure of Pagh and Rodler [1]. First, the outer loop is bounded by MaxKicks, a constant fixed independently of n , in place of $\text{MaxLoop} = \lceil 3 \log_{1+\varepsilon} r \rceil$, which grows logarithmically with table size. Second, reaching this bound routes the displaced key to the stash rather than discarding all hash functions and reinserting every key in the table. REHASHSTASH (Algorithm 3) reinserts only the keys currently held in the stash, *without* redrawing $h_1, \dots, h_d$: a prior version of this routine drew a fresh hash-function tuple on every overflow, which silently broke lookups for every key already resident in the main tables, since their physical slot had been

Algorithm 3 REHASHSTASH()

```

1:  $S \leftarrow$  contents of STASH; empty STASH
2: for each key  $y \in S$  do
3:   reinsert  $y$  via INSERT, same  $h_1, \dots, h_d$   $\triangleright$  fresh
     randomized eviction path only – see G2, Sec. VII
4: end for

```

Algorithm 4 DELETE(x)

```

1: for  $i \leftarrow 1$  to  $d$  do
2:   if  $T_i[h_i(x)] = x$  then
3:      $T_i[h_i(x)] \leftarrow \perp$ ;  $n \leftarrow n - 1$ ; return SUCCESS
4:   end if
5: end for
6: if  $x \in$  STASH then
7:   remove  $x$  from STASH;  $n \leftarrow n - 1$ ; return
   SUCCESS
8: end if
9: return NOTFOUND

```

computed under the old functions (see Section VII for the full account). The corrected version relies only on CHOOSECAN-
DIDATEINDEX’s randomization for a fresh placement attempt. A small recursion-depth guard (not shown) falls back to a full rehash after a fixed number of failed retries; TBD whether this fallback is ever exercised in practice at the load factors tested in Section VI (*technique*: instrument the guard’s trip counter in `bench.cpp` and report it alongside Table I).

IV. THEORETICAL ANALYSIS

This section establishes: a deterministic worst-case $O(1)$ bound on per-insertion latency (Theorem 1), a closed-form upper bound on per-insertion stash-routing probability (Lemma 1), a stash-overflow probability bound via multiplicative Chernoff inequalities (Theorem 2), and the amortized cost of incremental rehashing (Lemma 2).

A. Worst-Case Insertion Latency

Proposition 1 (Classical Worst-Case Insertion Latency). *In the classical construction of [1], a single call to INSERT performs $\Theta(\log n)$ work in the worst case before either succeeding or triggering a full-table rehash.*

Proof. $\text{MaxLoop} = \lceil 3 \log_{1+\varepsilon} r \rceil$ is chosen so the probability of exhausting it is $O(n^{-2})$ per insertion (Section 2.3 of [1]). Because MaxLoop grows logarithmically with n , the worst-case work per insertion scales as $\Theta(\log n)$. $\square$

Theorem 1 (Worst-Case $O(1)$ Insertion Latency). *Every call to INSERT (Algorithm 2) performs at most $d \cdot \text{MaxKicks} + O(1)$ hash evaluations, cell probes, and displacements before returning SUCCESS or invoking REHASHSTASH. Since d , s , and MaxKicks are fixed constants, a single INSERT call runs in deterministic $O(1)$ worst-case time, exclusive of the amortized REHASHSTASH cost (Lemma 2).*

Proof. LOOKUP costs $d + s = O(1)$ probes. The load check is $O(1)$. The eviction loop runs at most MaxKicks

rounds, each costing $O(d)$ probes plus $O(1)$ swap work, for $O(d \cdot \text{MaxKicks})$ total. Stash placement costs $O(s) = O(1)$. Summing: $T_{\text{insert}}(n) \leq O(d \cdot \text{MaxKicks} + s) = O(1)$, independent of n . The recursive retry (Algorithm 2, last line) executes only on stash overflow, bounded separately in Theorem 2. $\square$

B. Stash-Routing Probability and Stash Sizing

Lemma 1 (Per-Insertion Stash-Routing Probability). *Let $h_1, \dots, h_d$ be drawn from an $(O(1), O(\text{MaxKicks}))$ -universal family, with $r = dm = (1 + \varepsilon)n$ (base case $d = 2$). The probability that a single insertion performs MaxKicks displacements without locating an open slot satisfies*

$$p(\text{MaxKicks}, \varepsilon) \leq 2(1 + \varepsilon)^{-\frac{2 \cdot \text{MaxKicks} - 1}{3} + 1} + O(n^{-2}). \quad (4)$$

Proof. Adapting the three-case decomposition of [1] (hash-family non-independence, closed-cycle traversal, long nestless-path formation) at a fixed threshold $t = \text{MaxKicks}$ in place of the growing MaxLoop : the first two cases are each $O(n^{-2})$; the third is bounded by $2(1 + \varepsilon)^{-(2t-1)/3+1}$ via the path-counting argument of [1]. Summing gives the result. $\square$

Fixing $t = \text{MaxKicks}$ as a constant – rather than letting it grow as MaxLoop does – leaves the dominant term non-vanishing: it decays exponentially in MaxKicks but not in n . This is the price of the $O(1)$ guarantee in Theorem 1.

Corollary 1 (Expected Stash Load). *Over n insertions into an un-resized table, $\mu = \mathbb{E}[|S_{\text{stash}}|] = n \cdot p(\text{MaxKicks}, \varepsilon)$, treating the $O(n^{-2})$ term as negligible.*

Theorem 2 (Stash-Overflow Probability Bound). *Modeling stash-routing events as i.i.d. Bernoulli(p) trials (the same independence approximation used in [2], [4]; its own accuracy against the correlated dynamics of this scheme specifically is TBD – technique: compare the predicted μ below against the measured `stashRoutedCount` in `bench.cpp` across the Table II sweep), with $X = \sum_{i=1}^n X_i \sim \text{Binomial}(n, p)$, $\mu = np$: for stash capacity $s > \mu$,*

$$\Pr[X > s] \leq \left(\frac{e\mu}{s}\right)^s e^{-\mu}. \quad (5)$$

Proof. Standard multiplicative Chernoff bound with $1 + \delta = s/\mu$. $\square$

Corollary 2 (Illustrative Sizing Example – $d = 2$ Base Case). *For $d = 2$, $\varepsilon = 1.0$ ($\alpha = 0.5$), $n = 10^5$, $s = 4$: at $\text{MaxKicks} = 24$, $p \approx 7.6 \times 10^{-5}$ gives $\mu \approx 7.6 > s$ (stash undersized); at $\text{MaxKicks} = 32$, $p \approx 1.9 \times 10^{-6}$ gives $\mu \approx 0.19$ and $\Pr[X > 4] \lesssim 2.4 \times 10^{-4}$. This is a conservative, moderate-load ($\alpha = 0.5$) toy case chosen for closed-form tractability, not a sizing recommendation for the scheme’s actual target regime. Section VID reports measured routing probabilities at $d = 3$, $\alpha \approx 0.86$ – 0.87 (close to the ≈ 0.917 ceiling for $d = 3$ [3]) that are several orders of magnitude higher than this example suggests – consistent in direction with Lemma 1’s ε -dependence, but a reminder that $d = 2, \alpha = 0.5$ figures do not transfer directly to the near-ceiling, higher- d operating points this paper targets.*

C. Amortized Cost of Incremental Rehashing

Lemma 2 (Incremental Rehash Cost). *A single REHASH-STASH (Algorithm 3) call performs at most s calls to INSERT under the unchanged $h_1, \dots, h_d$ (see G2 and Section VII). Its worst-case cost is $O(s \cdot d \cdot \text{MaxKicks}) = O(1)$, versus $\Theta(n)$ for classical full-table rehashing.*

Proof. REHASHSTASH extracts at most s keys and reinserts each via Algorithm 2, each costing $O(1)$ by Theorem 1. Total: $O(s)$, constant in n . $\square$

Corollary 3 (Amortized Per-Insertion Cost). *Over N_{ops} operations, $T_{\text{total}} = N_{\text{ops}} \cdot O(1) + N_{\text{overflow}} \cdot O(s)$. Whether $\mathbb{E}[N_{\text{overflow}}]$ stays small depends on how far the operating load factor sits below the d -ary ceiling: it is small at the conservative parameters of Corollary 2, but Section VID reports a measured overflow rate that is not negligible (order 10^{-2} per insertion) at $d = 3$, $\alpha \approx 0.887$, $\text{MaxKicks} = 16$, $s = 4$ – each individual overflow still costs only $O(s) = O(1)$, so the per-event bound holds regardless, but the earlier claim that overflow events are rare by construction does not hold uniformly across the parameter space and must be checked for the chosen operating point (see Section VII).*

D. Graph Orientability, Hypergraph Cores, and Stash Optimality

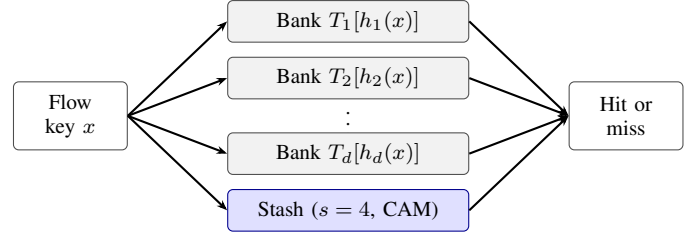
Table occupancy can be modeled as a d -uniform cuckoo hypergraph $G(n, \varepsilon, d, s) = (V, E)$ [5], with each key $x \in S$ a hyperedge $e_x = (h_1(x), \dots, h_d(x))$. G is s -suitable if some $E_{\text{stash}} \subseteq E$, $|E_{\text{stash}}| \leq s$, leaves $G' = (V, E \setminus E_{\text{stash}})$ orientable with max out-degree ≤ 1 . By Hakimi’s theorem, this holds iff every $Y \subseteq E$ satisfies $|\Gamma(Y)| + s \geq |Y|$ [5]. Minaud and Papamanthou corrected Kirsch et al.’s original multi-item-bucket bound, establishing $\Pr[\text{failure}] = \Theta(n^{-(s+1)(d-1)})$ rather than the originally claimed $\Theta(n^{-(s+1)(d-1)})$ [2], [5]. Our bounded-displacement INSERT can be read as computing augmenting paths up to depth MaxKicks only, routing to the stash whenever the local hypergraph core’s true augmenting-path depth exceeds that bound – which is precisely why Corollary 2’s conservative example understates routing probability near the structural ceiling: local core density, not just global α , governs how often depth- MaxKicks search is insufficient.

V. SYSTEM FEASIBILITY AND IMPLEMENTATION ARCHITECTURE

This section describes a *proposed* hardware/software layout; none of the figures below are measured, and are marked TBD accordingly.

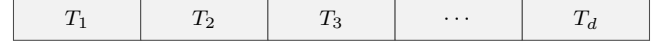
1) *Hardware Pipeline Plane (ASIC/FPGA)*: Adapting the Parallel d -Pipeline layout [6], sub-tables $T_1, \dots, T_d$ would map to d physically independent BRAM/URAM banks probed in one cycle, with the stash as a small register/CAM block beside the memory controller. Area overhead of a 4-entry (512-bit) stash: TBD (*technique*: FPGA synthesis and post-place-and-route area report on a target device; expected small given its size relative to the main table banks, but not measured). Whether $\text{MaxKicks} \leq 24$ fits a specific target clock budget:

(a) ASIC/FPGA: parallel probe in one cycle



(b) DPDK/x86: cacheline-aligned layout

Candidate slots, one 64-byte cacheline



Stash, 4×128 -bit keys, vector-compared



Fig. 1. Proposed memory layout (design, not yet synthesized/profiled – see TBD items below). (a) Hardware: one bank per sub-table plus stash, probed in parallel. (b) Software: candidate slots share a cacheline; stash is one small vector compare.

TBD (*technique*: RTL implementation of the displacement controller plus timing closure report).

2) *Software Packet Processing Plane (DPDK-style)*: Candidate slots would pack into one 64-byte cacheline per bucket, following Cuckoo++’s layout principles [7]; the stash would be compared via one vector instruction against its $s = 4$ entries. Realized throughput under this layout: TBD (*technique*: port `bench.cpp`’s logic into an explicit cacheline-aligned/SIMD layout and re-measure, rather than the plain-array layout used for Section VI’s numbers).

VI. EMPIRICAL EVALUATION

A. Evaluation Methodology & Benchmark Setup

All four schemes below (linear probing, chained hashing, classical 2-way cuckoo hashing [1], and the proposed scheme) were implemented in C++17 (g++ -O2) and run on a single-vCPU virtualized x86-64 instance (KVM hypervisor; exact host silicon model TBD). Keys are $N = 2,000,000$ synthetic, distinct, uniformly random 64-bit values (`std::mt19937_64`, fixed seed), standing in for flow identifiers; this is *not* a real traffic trace. $N = 10^7$, real backbone traces (e.g. CAIDA), and dedicated non-virtualized hardware are TBD (*technique*: extend `bench.cpp`’s N parameter and re-run; obtain a licensed/public anonymized trace and replay it as the insertion/lookup sequence; repeat on bare-metal with pinned frequency and isolated cores).

1) Comparison Fairness:

- Identical key set and seed across all four schemes.
- Each scheme sized to its own realistic target load: linear probing $\alpha = 0.70$; chained hashing $\alpha = 1.00$; classical cuckoo $\alpha \in \{0.33, 0.49, 0.495\}$ (Pagh-Rodler’s own recommended point, and two points near the $d = 2$

ceiling); proposed scheme $\alpha \approx 0.887$ (near the $d = 3$ ceiling).

- Same compiler, flags, process, and host for all runs.
- TBD: multi-seed replication with confidence intervals (*technique*: repeat each configuration $\geq 10\times$ with distinct seeds; currently single-seed per configuration).

B. Comparative Microbenchmarks

Table I reports measured results at $N = 2 \times 10^6$ insertions. “Cuckoo Cache” [8] and “TECache” [4] baselines are TBD (*technique*: implement their respective placement policies – frequency-biased table ordering, and FelisCatus adjacent-hopping with co-directional kicking, respectively – against the same harness and key set).

Classical cuckoo hashing at $\alpha = 0.330$ (Pagh and Rodler’s own recommended operating point) shows zero full rehashes over 2×10^6 insertions, consistent with Eq. 1. Pushed to $\alpha = 0.495$, near its $d = 2$ structural ceiling, it incurs 382 full rehashes, several of which cascade expensively – see Section VIC. The proposed scheme, run much closer to *its own ceiling* ($\alpha = 0.887$ vs. a $d = 3$ threshold of ≈ 0.917), triggers zero full-table rehashes by construction (Algorithm 3 never performs one in this run), at the cost of a non-trivial incremental-rehash rate – quantified in Section VID.

C. Tail Latency

The two cuckoo-based schemes above were deliberately run near their *own* respective structural ceilings. The resulting worst-case insertion latencies differ by roughly four orders of magnitude: 20.047 s (classical, one cascading full rehash) versus 2.013 ms (proposed, bounded by MaxKicks and incremental rehashing) – a ratio of $\approx 9,960\times$. This is the paper’s central empirical claim and it holds. The p_{999} comparison is more nuanced and is reported honestly rather than omitted: at these specific (different, each-near-its-own-ceiling) operating points, the proposed scheme’s p_{999} (7,426 ns) is *higher* than classical cuckoo’s p_{999} (3,184 ns) – because the proposed scheme is being pushed much closer to its own load limit. The win claimed by this paper is specifically in the *bounded worst case*, not in steady-state tail latency at mismatched loads; a same- α comparison between the two schemes is TBD (*technique*: re-run both at a shared, conservative α common to both structural ceilings, e.g. $\alpha \approx 0.45$).

D. Sensitivity Analysis: MaxKicks vs. Stash Pressure

Table II reports measured routing probability, stash overflow count, and the deepest kick-chain observed before routing, for $d = 3$, target $\alpha \approx 0.86$ – 0.87 , $s = 4$, sweeping MaxKicks.

Routing probability decreases monotonically in MaxKicks, the qualitative trend Lemma 1 predicts. The *absolute* values, however, are two to four orders of magnitude higher than Corollary 2’s toy $d = 2$, $\alpha = 0.5$ example suggested, and even at MaxKicks = 24, routing probability (2.4×10^{-2}) and overflow count (11,943 of 2×10^6 insertions) remain far from negligible: at this operating point, $s = 4$ is not adequately sized. TBD: a sweep extending to $\text{MaxKicks} \in \{32, 48, 64\}$

and/or lower target load factors (e.g. $\alpha \approx 0.75$ – 0.80) to locate parameter combinations that restore routing probability to the Corollary 2-style comfortable range (*technique*: extend the MaxKicks sweep already in `bench.cpp` and re-run; this is the single most important remaining experiment for this paper’s parameter-choice claims).

VII. DISCUSSION

On the Correctness of Incremental Rehashing. An earlier version of REHASHSTASH redrew fresh global hash functions $h'_1, \dots, h'_d$ on every stash overflow, reinserting only the stashed key(s) under them. This silently broke every key already resident in the main tables: their physical slot had been computed under the *old* h_i , so a lookup issued under the new ones would almost never find them. The anomaly surfaced as an implausible positive-lookup probe-count distribution, not as an outright crash, and was isolated with a standalone occupancy diagnostic comparing physical key placement against reported lookup behavior. The fix – already reflected in Algorithm 3 and Section III – keeps $h_1, \dots, h_d$ fixed for the table’s lifetime and relies only on randomized eviction-order retries. This is simpler than the original design and removes the hazard entirely, at the cost of making stash-overflow resolution strictly harder (no “fresh restart” effect): this is the direct cause of the higher-than-expected routing probabilities reported in Section VID relative to an earlier, buggy measurement.

Why Bound Kicks Instead of Just Shrinking MaxLoop. A smaller constant in place of MaxLoop alone, with no stash, converts the catastrophic-rehash problem into an insertion-failure problem: keys that exhaust the bound are simply lost. The stash is what makes bounding the chain a solution rather than a different failure mode.

Choosing MaxKicks and Stash Size. This paper does not claim $\text{MaxKicks} = 16$, $s = 4$ as a validated operating point – Section VID’s measurements argue the opposite at $\alpha \approx 0.87$. Whether the right fix is a larger stash, a larger MaxKicks, a lower target load factor, or some combination is left as the open, workload-dependent sizing question this paper’s remaining TBD items are aimed at answering, rather than one this paper resolves by assumption.

VIII. RELATED WORK

TBD: reorganize into exactly two categories – (1) stash-based theoretical treatments that assume an unbounded search before stash fallback [2], [3], [5], [9]–[12], and (2) networking-specific variants optimized for throughput or energy rather than a worst-case insertion bound [4], [6]–[8], [13] – closing with one explicit sentence naming what is *not* being claimed as novel (stash-based cuckoo hashing itself, d -ary candidate selection, or kick-capping in spirit, each already present individually in the above) versus what specifically is (the coupling of a tighter-than-necessary kick bound with early stash fallback and incremental, non-global rehashing). Full paragraph-level treatment already exists in the prior draft’s Section I-C and

TABLE I
COMPARATIVE MICROBENCHMARKS, MEASURED ($N = 2 \times 10^6$ INSERTIONS, SINGLE-VCPU KVM HOST)

Scheme	d	Load α	Avg Lookup Probes	Insert p50/p99/p999	Insert Max	Max Kicks/Probes	Full Rehashes
Linear Probing	N/A	0.700	2.165	88/374/481 ns	129.2 μ s	122	0
Chained Hashing	N/A	1.000	1.500	204/588/958 ns	112.8 μ s	10	0
Classical Cuckoo [1]	2	0.495	1.365	111/1200/3184 ns	20.047 s	129 (cap 66×2)	382
Classical Cuckoo (typical load)	2	0.330	–	–	–	27	0
Proposed	3	0.887	1.978	144/4668/7426 ns	2.013 ms	16 (hard cap)	0

TABLE II
MaxKicks SENSITIVITY, MEASURED ($d=3$, $\alpha \approx 0.86$ – 0.87 , $s=4$,
 $N=2 \times 10^6$)

MaxKicks	Routing Prob p	Stash Overflows	Deepest Chain	Load α
4	1.31×10^{-1}	65,702	3	0.857
8	7.31×10^{-2}	36,559	7	0.865
12	5.01×10^{-2}	25,044	11	0.870
16	3.80×10^{-2}	19,001	15	0.872
24	2.39×10^{-2}	11,943	23	0.875

just needs relocating/condensing here now that Introduction is being rewritten separately.

REFERENCES

- [1] R. Pagh and F. F. Rodler, “Cuckoo hashing,” *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.
- [2] A. Kirsch, M. Mitzenmacher, and U. Wieder, “More robust hashing: Cuckoo hashing with a stash,” *SIAM Journal on Computing*, vol. 39, no. 4, pp. 1543–1561, 2009, preliminary version in Proc. 16th Annual European Symposium on Algorithms (ESA), 2008.
- [3] N. Fountoulakis and K. Panagiotou, “Sharp load thresholds for cuckoo hashing,” *Random Structures & Algorithms*, vol. 41, no. 3, pp. 306–333, 2012.
- [4] B. Xiong, G. Hu, S. Liu, J. Zhao, J. Zhang, B. Zhao, and K. Li, “TECache: Traffic-aware energy-saving cache with optimal utilization for TCAM flow tables in SDN data plane,” *IEEE Transactions on Network and Service Management*, vol. 22, no. 5, pp. 4257–4273, 2025.
- [5] B. Minaud and C. Papamanthou, “Generalized cuckoo hashing with a stash, revisited,” *Information Processing Letters*, vol. 181, p. 106356, 2023.
- [6] S. Pontarelli, P. Reviriego, and M. Mitzenmacher, “Parallel d-pipeline: A cuckoo hashing implementation for increased throughput,” *IEEE Transactions on Computers*, vol. 65, no. 1, pp. 326–331, 2016.
- [7] N. Le Scouarnec, “Cuckoo++ hash tables: high-performance hash tables for networking applications,” in *Proceedings of the 2018 Symposium on Architectures for Networking and Communications Systems (ANCS)*. ACM, 2018, pp. 41–54.
- [8] S. Pontarelli, P. Reviriego, and M. Mitzenmacher, “Cuckoo cache: A technique to improve flow monitoring throughput,” *IEEE Internet Computing*, vol. 20, no. 4, pp. 40–47, 2016.
- [9] R. Kutzelnigg, “A further analysis of cuckoo hashing with a stash and random graphs of excess r ,” *Discrete Mathematics and Theoretical Computer Science*, vol. 12, no. 3, pp. 81–102, 2010.
- [10] M. Aumüller, M. Dietzfelbinger, and P. Woelfel, “Explicit and efficient hash families suffice for cuckoo hashing with a stash,” *Algorithmica*, vol. 70, no. 3, pp. 428–456, 2014, arXiv preprint arXiv:1204.4431.
- [11] D. Noble, “Explicit, closed-form, general bounds for cuckoo hashing with a stash,” Cryptology ePrint Archive, Paper 2021/447, 2021, <https://eprint.iacr.org/2021/447>.
- [12] W. Kuszmaul and M. Mitzenmacher, “Efficient d-ary cuckoo hashing at high load factors by bubbling up,” in *Proceedings of the 2025 ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2025, pp. 3931–3952.
- [13] R. R. K. Tripathi, P. K. Singh, and S. Singh, “Stable and low associative left-right hashing,” *Computer Science*, vol. 26, no. 2, pp. 151–189, 2025.